

Laboration 3: Watts-Strogatz Networks and Markov-Chain Convergence

Group Supermodels

Vootele Mets

Juan Rodriguez

Marco Malosti

Björk Lucas

Sofia Fernandes

David Weingut

May 15, 2026



UPPSALA
UNIVERSITET

Contents

1	Problem Statement	2
2	Watts-Strogatz Networks	3
2.1	Theory	3
2.1.1	Degrees and degree distribution	3
2.1.2	Connected components	3
2.1.3	Shortest distance, diameter and average path length	4
2.1.4	Clustering Coefficient	4
2.2	Methods	6
2.2.1	Degrees and degree distribution	6
2.2.2	Connected components	6
2.2.3	Diameter and average path length	6
2.2.4	Clustering Coefficient	7
2.3	Results	8
2.3.1	Degrees and degree distribution	8
2.3.2	Connected components	9
2.4	Diameter and Average Shortest Path Length	10
2.4.1	Clustering Coefficient	10
3	Markov-Chain Convergence	11
3.1	Theory	11
3.1.1	Stationary distribution	13
3.1.2	Error dynamics	15
3.1.3	Spectral Decomposition	16
3.1.4	Estimated time to reach a tolerance	17
3.2	Methods	19
3.3	Results	20
4	Conclusion	24
4.1	Synthesis	24
5	Use of LLM tools	26

1. Problem Statement

In this lab report two topics are studied: Watts-Strogatz networks and Markov-chain convergence.

A Watts-Strogatz network is a type of random undirected graph (a collection of nodes and connecting vertices) that is used to model social networks. In the first part of this report Watts-Strogatz networks are studied using large statistical sets and then varying different parameters of the network. The studied network characteristics are the node degree distributions, statistics of connected components, shortest path distance, diameter, average path length and the clustering coefficient.

A Markov chain is a stochastic process that describes a sequence of possible events in which the probability of each event depends only on the state attained in the previous event. In the second part of this report Markov-chain convergence to its stationary distribution is studied. First the theory behind Markov chain convergence, stationary distribution, error dynamics and spectral decomposition is explained. Then the convergence to a stationary distribution is numerically computed.

Based on the findings of the two topics' questions about both Watts-Strogatz networks and Markov chains are answered in the Synthesis section of this report.

2. Watts-Strogatz Networks

2.1. Theory

Watts-Strogatz graphs are a type of network that are constructed as follows:

- N nodes are arranged on a ring
- In the beginning all nodes are connected to their $k/2$ closest neighbours to both sides (every node has degree k)
- Each edge is rewired with a probability of p (no self-loops or duplicate edges allowed)

Independent of p , all graphs with the same N and k will have the same number of edges. However, these edges will be placed different depending on p as a rewired edge can be placed anywhere and not only between nodes situated close to each other. For small $p \neq 0$ the rewiring will lead to a few long-range shortcuts. For $p = 1$ all edges are rewired and we get a completely random graph. Below is a visualisation of the graph.

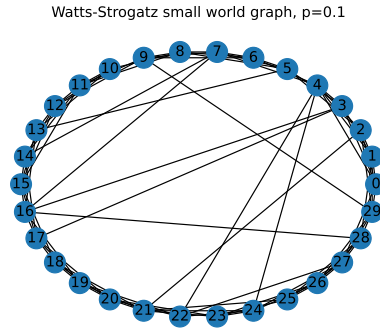


Figure 1. A Watts-Strogatz graph with $N = 30$, $k = 6$ and $p = 0.1$. The lines inside the circle are the random edges generated with a non-zero p .

2.1.1. Degrees and degree distribution

For a graph node i the degree d_i describes the number of edges incident to that node. It is also possible to calculate the average degree for the whole graph using

$$\bar{d} = \frac{1}{N} \sum_{i=1}^N d_i \quad (2.1)$$

where N is the total number of nodes. In the case of a Watts-Strogatz graph the average degree is equal to k as initially every node is connected to k other nodes. At $p = 0$ there are no random connections added and every node has exactly k connections and so for $p = 0$ every node has the degree is $d_i = k$. As p is increased, random connections appear and thus d_i starts to vary between nodes but the mean d_i would still be equal to k .

2.1.2. Connected components

An undirected graph is called connected if every node can be connected to every other node by a path. If a graph is disconnected then a connected subgraph of the original

graph is called a connected component. A connected component can be studied by computing different distance quantities for it such as the shortest path length, the radius and the diameter.

2.1.3. Shortest distance, diameter and average path length

In a Watts-Strogatz small world graph, edges connect two arbitrary nodes i and j through a path, a sequence of edges. A graph is connected if any i can be reached from any j through any path of any length. With the assumption that every graph is connected, define the diameter and average path length. Begin with $\text{dist}(i, j)$, which is the shortest-path distance between i and j , i.e. the least number of edges which connect the two nodes. The largest possible shortest path between two nodes in the graph is the diameter, written as

$$\text{diam}(G) = \max_{i,j} \text{dist}(i, j). \quad (2.2)$$

The average path length is instead the average shortest path length between i and j and is defined as follows:

$$\ell(G) = \frac{1}{N(N-1)} \sum_{i \neq j} \text{dist}(i, j) \quad (2.3)$$

A graph can also be disconnected if there are nodes or sets of nodes that cannot be reached from any other node. The diameter and average path length would still be defined in the same way as for a connected graph, but the numerical value would be different once computed.

2.1.4. Clustering Coefficient

The local clustering coefficient measures how many neighbouring nodes of one node are connected among themselves. Using the example of friendship, it describes how many of your friends are friends with your other friends.

Mathematically, the local clustering coefficient is the ratio of the number of edges between the neighbours and the possible number of edges between the neighbours. Thus, a clustering coefficient of 1 means high clustering, all neighbours are connected, while 0 means no clustering or none of a node's neighbours are connected. Let d_i denote the degree, or number of neighbours, of node i . Then the maximum number of connections between node i 's neighbours is

$$\frac{d_i(d_i - 1)}{2}$$

With e_i denoting the number of these edges that are present, the local clustering coefficient becomes

$$C_i = \frac{2e_i}{d_i(d_i - 1)}$$

If a node has only one or no neighbour ($d_i < 2$), the local clustering coefficient is automatically set to $C_i = 0$.

A more general number than the local clustering coefficient is the average clustering coefficient, which averages over all nodes of a graph.

$$C(G) = \frac{1}{N} \sum_{i=1}^N C_i$$

75 where G denotes the graph and N is the number of nodes. This number will be calculated
76 for all graphs and plotted together with the average path length.

2.2. Methods

The package `NetworkX` offers the function `watts_strogatz_graph` to make a Watts-Strogatz graph. Conveniently, this function takes N , k and p as arguments. Using this function we created all graphs for this exercise. Thus, to run our code, the package `NetworkX` needs to be installed. We used the suggested values of $N = 200$ and $k = 6$ for all graphs, but will run several simulations while varying p . The different values of the probability will be picked from the following list: $[0.0, 0.001, 0.003, 0.01, 0.03, 0.10, 0.30, 1.0]$.

The graph can also be plotted using the `draw_circular()` or `draw_networkx()`, which takes in the Watts-Strogatz graph and generates a circular plot or an arbitrary plot. Typically, the graph is shown as a circle and this is how it was presented in figure 1.

2.2.1. Degrees and degree distribution

To study how the distribution of the degree d_i of a Watts-Strogatz graph changes by varying p value a python script `WS_1_degrees.py` was written. It can be run using the command `python3 WS_1_degrees.py`. The script creates a Watts-Strogatz graphs with fixed parameters $N = 200$ and $k = 6$ and a variable parameter p that is varied from 0.0 to 1.0. For each p value 1000 separate instances of the graph are created and the degree of each node is studied. Then the results over the 1000 instances are averaged and so are created degree distributions for each p value. Finally histograms are created for each p value and saved as a file `WS_1_degrees.png`. The script utilizes the following Python packages: `NetworkX`, `NumPy` and `matplotlib`.

2.2.2. Connected components

To study the connected components of a Watts-Strogatz graph a python script `WS_2_connected.py` was written. It can be run using the command `python3 WS_2_connected.py`. The script creates a Watts-Strogatz graphs with fixed parameters $N = 200$ and $k = 2$ and a variable parameter p that is varied from 0.0 to 1.0. For each p value 1000 separate instances of the graph are created. Then the code logs if the graph is fully connected or not. For every instance the number of connected components and the size of the largest connected component is recorded. For disconnected graphs the distance quantities shortest path length, radius and diameter are also computed. These quantities are then averaged over the 1000 instances. Finally a graph is created for each quantity showing its relation to p . All the graphs are saved in the file `WS_2_connected_plot.png`.

2.2.3. Diameter and average path length

The same Python package `NetworkX` has many useful functions. Once the Watts-Strogatz graph has been generated using the `nx.watts_strogatz_graph` function, the following functions are then used: `nx.diameter()` and `nx.average_shortest_path_length()`. As the names of the functions indicate, the `nx.diameter()` returns the diameter (as defined in equation 2.2) and the latter function computes the average path length (as defined in equation 2.3). In the case of an disconnected graph, the diameter and average path length would be computed for the largest connected component of the graph.

The computations are repeated 1000 times for each value of p , with a new seed each times. For every iteration, a new graph is generated and the relevant values are saved in a `NumPy`-array. This will then be used to produce the results visible in section 2.4.

2.2.4. Clustering Coefficient

The package `NetworkX` has the function `clustering`, with which the clustering of a graph can be calculated. The average shortest path length is calculated using the function `nx.average_shortest_path_length`. In our code we iterate over the different values of p ($p = [0, 0.001, 0.003, 0.01, 0.03, 0.1, 0.3, 1]$) as well as 1000 different seeds (1-1000). This way the results of our calculations are reproducible.

The function `networkx.clustering` outputs a dictionary with the local clustering coefficients of each node. For each graph, we take the average of these local clustering coefficients and append them to a list for the current p . After iterating over all 1000 seeds, the average for the current p -value is taken and saved to another list. Additionally, the standard deviation of the clustering coefficients is calculated using `numpy` and saved to yet another list. Then we continue with the next value of p .

After iterating over all p values, the averaged values and their standard deviation are plotted using `plt.errorbar`. All values are normalised by the value at $p = 0$. The whole code is timed, as we were afraid it would take too long to execute it.

To reproduce Figure 5 you need to run `clustering_coeff.py`. The required packages are `NetworkX`, `numpy`, `matplotlib` and `time`.

2.3. Results

2.3.1. Degrees and degree distribution

The results of degree distribution study are given in figure 2.

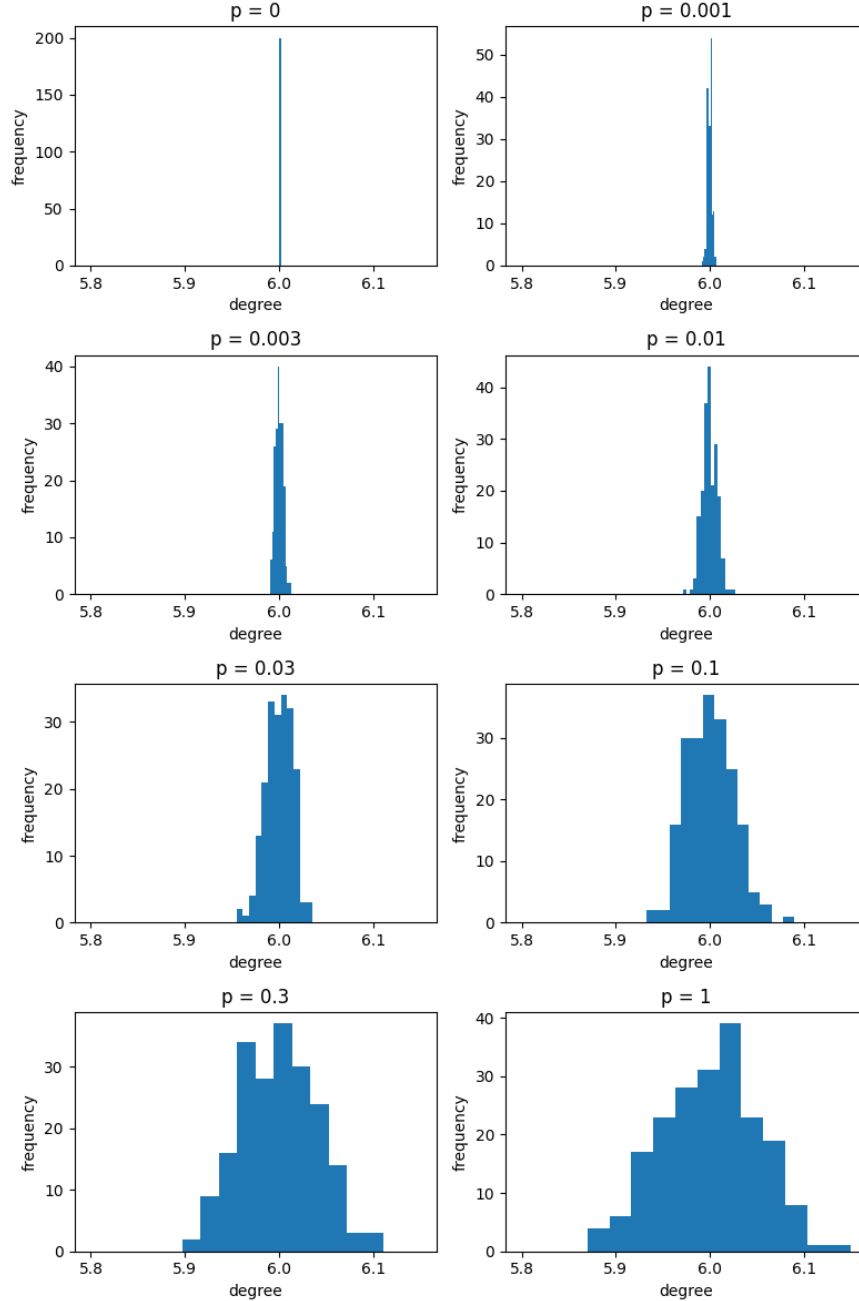


Figure 2. Degree distribution for a Watts-Strogatz graph with values $N = 200$ and $k = 6$ for different p values.

The graph shows that at $p = 0$ every node has $d = k$ as was predicted in the theory. As p gets larger a gaussian shaped distribution starts to emerge. At higher p values the distribution gets wider, showcasing a more randomized graph but the mean value of the distribution still stays around 6.0 which is the fixed k value.

2.3.2. Connected components

The connected component statistics for a Watts-Strogatz graph with $N = 200$ and $k = 2$ are shown in figure 3.

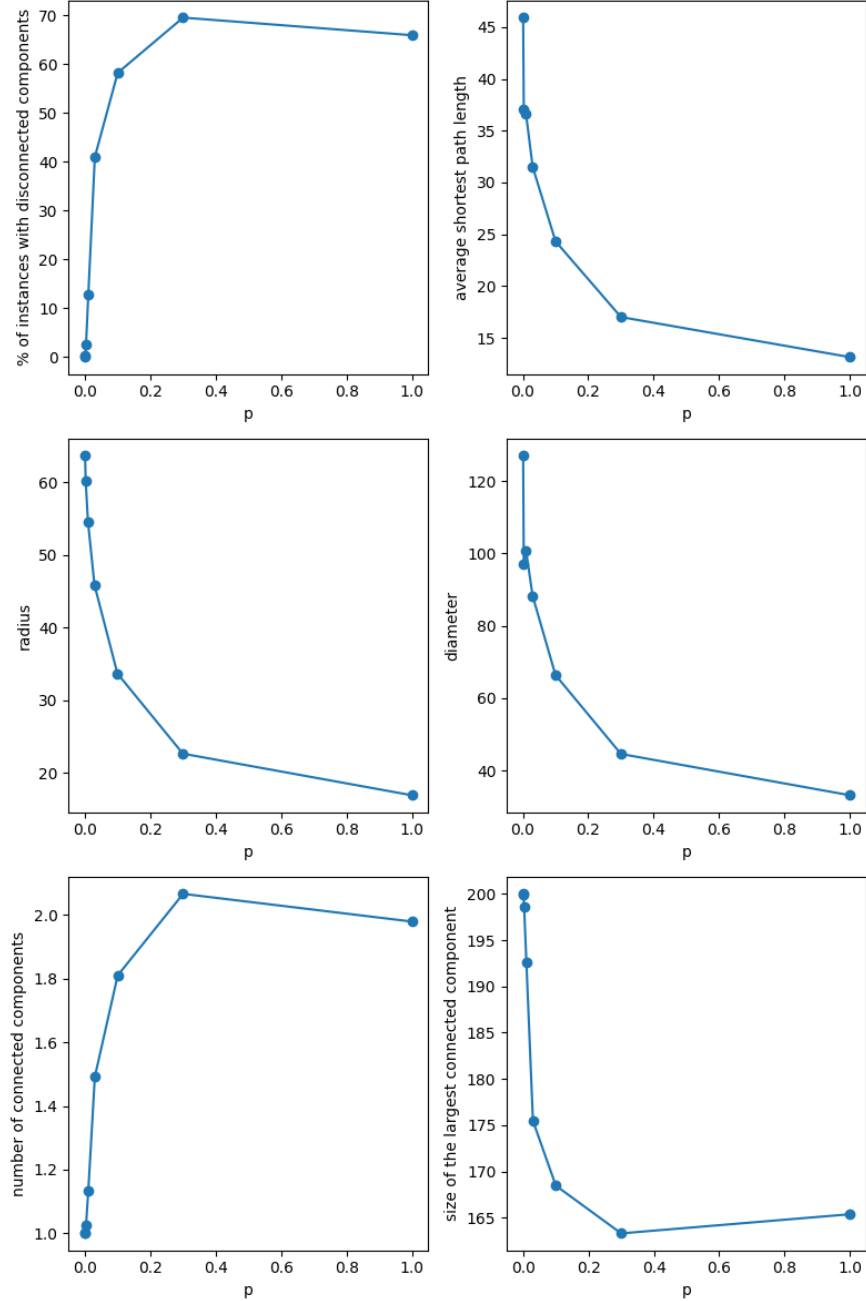


Figure 3. Component statistics and distance quantities for a Watts-Strogatz graph with values $N = 200$ and $k = 2$. x-axis shows the p value and y-axis the studied component statistic.

From the figure it is evident that the chance of not having a connected graph increases when increasing p but for $p = 1$ slightly decreases. Shortest path length along with radius and diameter all decrease. At $p = 0$ there is only 1 connected component as there are no random connections and each node is connected to two neighbors. The

amount of connected components rises with increasing p reaching a peak at around 2 for $p = 0.3$ and slightly decreases for $p = 1.0$

Versions of `WS_2_connected.py` with $k = 6$ and $k = 4$ were also investigated but in those cases all the created graph instances were fully connected for every p value and so did not yield interesting statistics.

2.4. Diameter and Average Shortest Path Length

The saved values of the diameter and average shortest path length make up two NumPy-arrays with a total of 16000 values. Plotting these is quite messy, so the mean and standard deviation of the 1000 iterations for each value of p are computed. These are shown in the following figure:

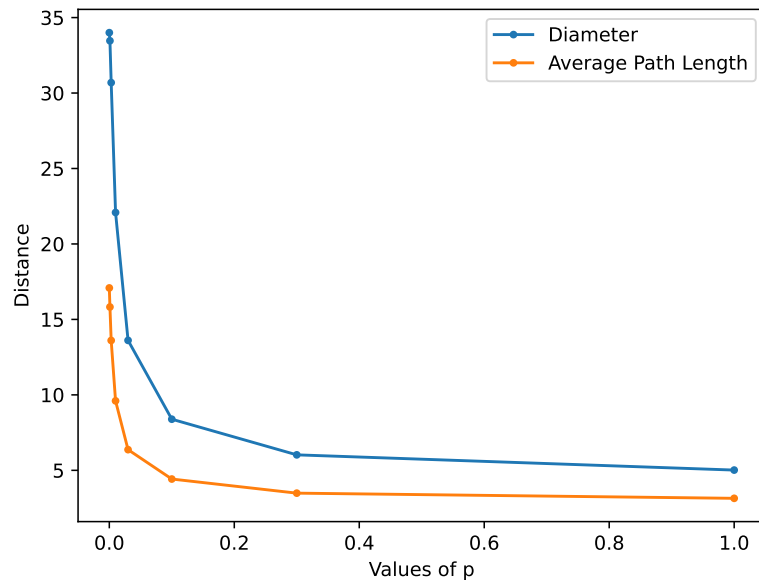


Figure 4. The mean values of the diameter and average shortest path length for different values of p .

It is visible that the quantity of edges required to reach any node on the graph from any starting node diminishes drastically with higher quantity of random connections. In other words, the paths are generally shorter with higher p , and the world feels smaller.

2.4.1. Clustering Coefficient

Figure 5 shows the average clustering coefficient and the average shortest path length for different p , normalised by the value at $p = 0$. For the values $p = 0.03$ and $p = 0.1$ we can observe the small-world effect. The clustering coefficient retains relatively high values at around 0.9 for $p = 0.03$ and 0.75 for $p = 0.1$. At the same time, the average shortest path length has dropped to 0.4 and 0.25 of the value at $p = 0$. Thus, we can still see a substantial clustering while the average shortest path length has already decreased.

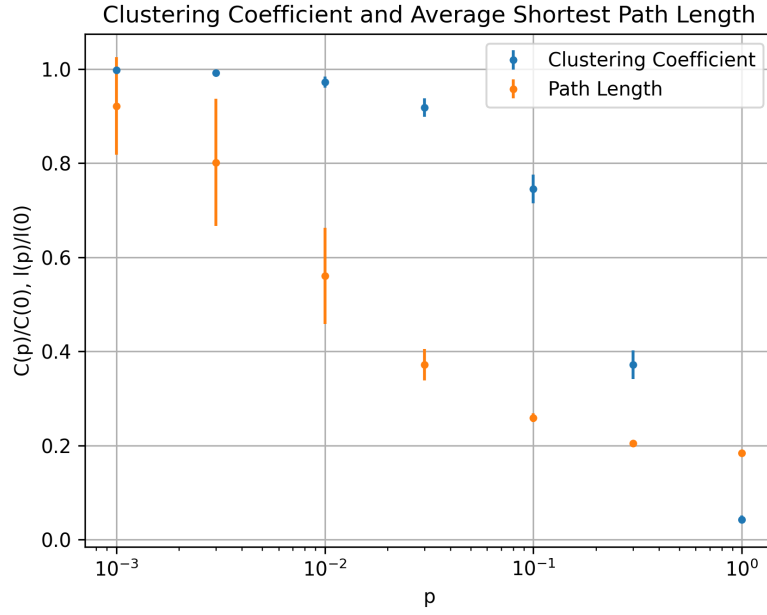


Figure 5. Average clustering coefficient and shortest path length for different values of p

3. Markov-Chain Convergence

3.1. Theory

A Markov chain is a mathematical framework typically categorized as a collection of random variables that progresses starting with one state then onto the next based on probabilistic guidelines. The likelihood of changing to a specific state depends exclusively on the present state and elapsed time and not on the series of states that have preceded it; it does not retain memory. This progression of states of a framework is called transitions [5]. In our case the Markov chain is time independent. One application of Markov chains is very present while doing this project, for example. The PageRank algorithm is one of the most used applications. Originally, it was developed to rank web pages in search engines. In this context, each web page can be considered a state, and each link from one page to another represents a possible transition. Pages with a higher stationary probability are interpreted as more important, because, in the long run, someone would visit them more often than the others [4].

The convergence of a Markov chain to equilibrium can be studied through the eigenvalues of its transition matrix. Since in our case P is a stochastic matrix, the largest eigenvalue is

$$\lambda_1 = 1.$$

This eigenvalue corresponds to the stationary distribution π , which satisfies

$$P\pi = \pi.$$

The remaining eigenvalues describe how fast the non-stationary parts of the initial distribution decay. If the eigenvalues are ordered by decreasing modulus,

$$\lambda_1 = 1, \quad |\lambda_2| \geq |\lambda_3| \geq \dots \geq |\lambda_N|,$$

190 then the second-largest eigenvalue modulus is defined as

$$\mu = |\lambda_2|.$$

191 The value of μ controls the asymptotic rate of convergence to stationarity [3]. If μ is
192 close to 1, then the convergence is slow, because the dominant non-stationary component
193 decays slowly. If μ is farther from 1, then the convergence is faster.

194 It is therefore useful to define the spectral gap as

$$g = 1 - \mu.$$

195 A larger spectral gap means faster convergence, while a smaller spectral gap means
196 slower convergence.

197 In this exercise, we use the column-stochastic convention [1]. This means that if x_t is
198 the probability distribution at time t , then the dynamics are given by

$$x_{t+1} = Px_t.$$

199 Each column of P must sum to 1, ensuring that the total probability is preserved at
200 each step.

201 We also consider the lazy variants [2]

$$P_\alpha = \alpha I + (1 - \alpha)P_0,$$

202 with $\alpha = 0.5$ and $\alpha = 0.8$. Increasing α means that the chain has a larger probability
203 of remaining in its current state.

204 The lazy variants have the same stationary distribution as P_0 . If λ is an eigenvalue of
205 P_0 , then

$$\alpha + (1 - \alpha)\lambda$$

206 is the corresponding eigenvalue of P_α . Therefore, larger values of α move the nontrivial
207 eigenvalues closer to 1, which increases μ , decreases the spectral gap, and slows down
208 convergence.

$$P_0 = \begin{pmatrix} 0.60 & 0.20 & 0.15 & 0.10 \\ 0.20 & 0.50 & 0.20 & 0.20 \\ 0.10 & 0.20 & 0.45 & 0.20 \\ 0.10 & 0.10 & 0.20 & 0.50 \end{pmatrix}.$$

209 Then repeat the experiment for the lazy variants

$$P_\alpha = \alpha I + (1 - \alpha)P_0, \quad \alpha = 0.5, 0.8.$$

$$P_\alpha = \alpha \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} + (1 - \alpha) \begin{pmatrix} 0.60 & 0.20 & 0.15 & 0.10 \\ 0.20 & 0.50 & 0.20 & 0.20 \\ 0.10 & 0.20 & 0.45 & 0.20 \\ 0.10 & 0.10 & 0.20 & 0.50 \end{pmatrix}.$$

$$P_{0.5} = 0.5 \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} + 0.5 \begin{pmatrix} 0.60 & 0.20 & 0.15 & 0.10 \\ 0.20 & 0.50 & 0.20 & 0.20 \\ 0.10 & 0.20 & 0.45 & 0.20 \\ 0.10 & 0.10 & 0.20 & 0.50 \end{pmatrix} = \begin{pmatrix} 0.80 & 0.10 & 0.075 & 0.05 \\ 0.10 & 0.75 & 0.10 & 0.10 \\ 0.05 & 0.10 & 0.725 & 0.10 \\ 0.05 & 0.05 & 0.10 & 0.75 \end{pmatrix}.$$

$$P_{0.8} = 0.8 \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} + 0.2 \begin{pmatrix} 0.60 & 0.20 & 0.15 & 0.10 \\ 0.20 & 0.50 & 0.20 & 0.20 \\ 0.10 & 0.20 & 0.45 & 0.20 \\ 0.10 & 0.10 & 0.20 & 0.50 \end{pmatrix} = \begin{pmatrix} 0.92 & 0.04 & 0.03 & 0.02 \\ 0.04 & 0.90 & 0.04 & 0.04 \\ 0.02 & 0.04 & 0.89 & 0.04 \\ 0.02 & 0.02 & 0.04 & 0.90 \end{pmatrix}.$$

210 Given what was stated above, for a general Markov chain, if you encounter an eigenvalue
 211 very close to -1 , the chain will converge slowly. This convergence also comes with
 212 oscillations because of the associated error component, which is associated with an
 213 eigenvalue, evolving approximately as

$$\lambda^t \approx (-1)^t |\lambda|^t.$$

214 The distribution, therefore, oscillates between states, instead of smoothly approaching
 215 the stationary distribution.

216 Now, replacing the original chain by the lazy chain,

$$P_{\text{lazy}} = \frac{1}{2} \left[\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} + \begin{pmatrix} 0.60 & 0.20 & 0.15 & 0.10 \\ 0.20 & 0.50 & 0.20 & 0.20 \\ 0.10 & 0.20 & 0.45 & 0.20 \\ 0.10 & 0.10 & 0.20 & 0.50 \end{pmatrix} \right].$$

$$P_{\text{lazy}} = \frac{1}{2} \begin{pmatrix} 1.60 & 0.20 & 0.15 & 0.10 \\ 0.20 & 1.50 & 0.20 & 0.20 \\ 0.10 & 0.20 & 1.45 & 0.20 \\ 0.10 & 0.10 & 0.20 & 1.50 \end{pmatrix}.$$

$$P_{\text{lazy}} = \begin{pmatrix} 0.80 & 0.10 & 0.075 & 0.05 \\ 0.10 & 0.75 & 0.10 & 0.10 \\ 0.05 & 0.10 & 0.725 & 0.10 \\ 0.05 & 0.05 & 0.10 & 0.75 \end{pmatrix}.$$

217 3.1.1. Stationary distribution

218 A Markov chain that is irreducible and aperiodic with finite state space has a unique
 219 stationary distribution π , which is a probability vector such that

$$P\pi = \pi.$$

220 Additionally, the transition probabilities converge to a steady state when the number of
221 steps goes to infinity, in the sense that

$$\lim_{m \rightarrow \infty} p_{ij}^{(m)} = \pi_i \quad \text{for all } i, j \in S.$$

222 Observing matrix P_0 , we can directly infer that there are no negative values. To satisfy

$$\sum_i (P_0)_{ij} = 1,$$

223 we have that

$$0.60 + 0.20 + 0.10 + 0.10 = 1;$$

$$224 \quad 0.20 + 0.50 + 0.20 + 0.10 = 1;$$

$$225 \quad 0.15 + 0.20 + 0.45 + 0.20 = 1;$$

$$226 \quad 0.10 + 0.20 + 0.20 + 0.50 = 1.$$

227 Satisfying both conditions for a valid transition matrix.

228 Computing the stationary distribution π as the eigenvector associated with the eigen-
229 value $\lambda = 1$, π satisfies

$$P_0 \pi = \pi.$$

230 Equivalently, we can formulate the eigenvalue problem as

$$(P_0 - I)\pi = 0.$$

231 We can solve this considering,

$$\pi = \begin{pmatrix} \pi_1 \\ \pi_2 \\ \pi_3 \\ \pi_4 \end{pmatrix}.$$

232 So,

$$P_0 - I = \begin{pmatrix} -0.40 & 0.20 & 0.15 & 0.10 \\ 0.20 & -0.50 & 0.20 & 0.20 \\ 0.10 & 0.20 & -0.55 & 0.20 \\ 0.10 & 0.10 & 0.20 & -0.50 \end{pmatrix}.$$

233 The system to solve is then

$$\begin{cases} -0.40\pi_1 + 0.20\pi_2 + 0.15\pi_3 + 0.10\pi_4 = 0, \\ 0.20\pi_1 - 0.50\pi_2 + 0.20\pi_3 + 0.20\pi_4 = 0, \\ 0.10\pi_1 + 0.20\pi_2 - 0.55\pi_3 + 0.20\pi_4 = 0, \\ 0.10\pi_1 + 0.10\pi_2 + 0.20\pi_3 - 0.50\pi_4 = 0. \end{cases}$$

234 Since π is a probability distribution, we have to impose the normalization condition

$$\pi_1 + \pi_2 + \pi_3 + \pi_4 = 1.$$

235 Solving this system gives

$$\pi = \begin{pmatrix} \frac{149}{532} \\ \frac{2}{7} \\ \frac{61}{266} \\ \frac{109}{532} \end{pmatrix} \approx \begin{pmatrix} 0.2801 \\ 0.2857 \\ 0.2293 \\ 0.2049 \end{pmatrix}.$$

236 We can then check that the entries also sum to 1:

$$0.2801 + 0.2857 + 0.2293 + 0.2049 \approx 1.$$

237 This way, we can verify that the distribution is stationary,

$$P_0\pi = \begin{pmatrix} 0.60 & 0.20 & 0.15 & 0.10 \\ 0.20 & 0.50 & 0.20 & 0.20 \\ 0.10 & 0.20 & 0.45 & 0.20 \\ 0.10 & 0.10 & 0.20 & 0.50 \end{pmatrix} \begin{pmatrix} 0.2801 \\ 0.2857 \\ 0.2293 \\ 0.2049 \end{pmatrix} \approx \begin{pmatrix} 0.2801 \\ 0.2857 \\ 0.2293 \\ 0.2049 \end{pmatrix}.$$

238 Concluding,

$$P_0\pi \approx \pi,$$

239 so π is the stationary distribution of the Markov chain.

240 The stationary distribution π is an equilibrium for the Markov-chain dynamics because
241 it remains unchanged when the transition matrix is applied, which means

$$P\pi = \pi.$$

242 So, if the chain has a distribution π at time t , it will also have distribution π at time $t + 1$.
243 However, this does not mean that an individual realization of the chain remains fixed in
244 one state, because a single trajectory may continue to move between states. So, in this
245 case, stationarity means that the overall probability of being in a state remains constant
246 over time. This probability can be observed from individual chains by averaging over a
247 large number of individual trajectories or a single very long trajectory.

248 **3.1.2. Error dynamics**

249 Considering x_0 as an initial probability distribution, with stationary distribution

$$P\pi = \pi,$$

250 and initial state x_0 the evolution can be described by

$$x_t = P^t x_0.$$

251 We can set

$$e_t = x_t - \pi. \tag{3.1}$$

252 With the assumption of the stationary distribution, we know that if the chain has a
253 distribution π at time t , it will also have distribution π at time $t + 1$. So, in order to
254 calculate the following term,

$$e_{t+1},$$

255 we just need to apply the transition matrix. This can be shown by putting the update
256 equation,

$$x_{t+1} = Px_t$$

257 in equation 3.1. We obtain

$$e_{t+1} = Px_t - \pi.$$

258 Since π is stationary, we have $\pi = P\pi$. Therefore,

$$e_{t+1} = Px_t - P\pi.$$

259 Factoring out P , we get

$$e_{t+1} = P(x_t - \pi).$$

260 Since $e_t = x_t - \pi$, this gives

$$e_{t+1} = Pe_t.$$

261 Applying this relation repeatedly gives

$$e_1 = Pe_0,$$

262

$$e_2 = Pe_1 = P^2e_0,$$

263

$$e_3 = Pe_2 = P^3e_0.$$

264

$$\vdots$$

265 Therefore, as a general rule, we obtain

$$e_t = P^te_0.$$

266 Given this general rule, it is clear that the convergence is determined by how the powers
267 of P act on the initial error e_0 . This means that if P^te_0 becomes small as t increases,
268 then e_t also becomes small, resulting in x_t approaching π . One can conclude that the
269 Markov chain converges to stationarity when the initial error is progressively reduced by
270 repeated multiplications of P . To analyze this convergence we employ spectral analysis.

271 3.1.3. Spectral Decomposition

272 The spectral decomposition follows the argument given in the lab instructions [2]. As-
273 suming diagonalizable P , its eigenvalues can be ordered by their moduli,

$$\lambda_1 = 1, \quad |\lambda_2| \geq |\lambda_3| \geq \dots \geq |\lambda_N|.$$

274 The eigenvalue $\lambda_1 = 1$ corresponds to the stationary distribution π , because applying P
275 to π leaves it unchanged,

$$P\pi = \pi.$$

276 The initial error is defined as

$$e_0 = x_0 - \pi.$$

277 This error measures the difference between the initial distribution and the stationary
278 distribution. Since both x_0 and π are probability distributions, their entries both sum
279 to 1. Therefore, the entries of e_0 will sum to zero

$$\sum_i e_0(i) = 1 - 1 = 0.$$

280 This means that e_0 only describes the part of the initial distribution that must decay
 281 over time, not the stationary part that remains unchanged. Therefore, we can write the
 282 initial error as a linear combination of the remaining eigenvectors,

$$e_0 = c_2 v_2 + c_3 v_3 + \cdots + c_N v_N.$$

283 Using the error dynamics, like seen before,

$$e_t = P^t e_0,$$

284 we obtain

$$e_t = P^t (c_2 v_2 + c_3 v_3 + \cdots + c_N v_N).$$

285 Since each eigenvector satisfies the condition

$$P v_i = \lambda_i v_i,$$

286 we have that

$$P^t v_i = \lambda_i^t v_i.$$

287 Therefore,

$$e_t = c_2 \lambda_2^t v_2 + c_3 \lambda_3^t v_3 + \cdots + c_N \lambda_N^t v_N.$$

288 This expression shows that each non-stationary component of the initial error decays
 289 according to the corresponding eigenvalue. The slowest-decaying term is controlled by
 290 the largest modulus among the nontrivial eigenvalues,

$$\mu = |\lambda_2|.$$

291 So, for a suitable constant K depending on the initial condition and on the eigenvectors,
 292 we have the estimate

$$\|x_t - \pi\| \leq K \mu^t.$$

293 This explains why the second-largest eigenvalue modulus controls the convergence rate.

294 **3.1.4. Estimated time to reach a tolerance**

295 Following [2] we suppose an error to be smaller than a tolerance ε :

$$\|x_t - \pi\| \leq \varepsilon.$$

296 Using the estimate

$$\|x_t - \pi\| \lesssim K \mu^t,$$

297 we require

$$K \mu^t \lesssim \varepsilon.$$

298 Dividing by K , we get

$$\mu^t \lesssim \frac{\varepsilon}{K}.$$

299 Taking the logarithms gives

$$t \log(\mu) \lesssim \log\left(\frac{\varepsilon}{K}\right).$$

300 Since $0 < \mu < 1$ for $\alpha < 1$, we have $\log(\mu) < 0$. Dividing by this negative number
 301 reverses the inequality direction:

$$t \gtrsim \frac{\log(K/\varepsilon)}{-\log(\mu)}.$$

302 When μ is close to 1, we can use the approximation

$$-\log(\mu) \approx 1 - \mu = g,$$

303 where g is the spectral gap. Therefore,

$$t \sim \frac{1}{g}$$

304 where we drop the constants of the chosen tolerance and unknown but constant mul-
305 tiplicative factor K . This verifies the statement given in the beginning of this section,
306 that a larger spectral gap results in a faster convergence, while a smaller spectral gap
307 results in slower convergence.

3.2. Methods

This subsection serves to explain the numerical methods employed to perform the numerical verification of the theoretical part outlined previously. The whole program makes heavy use of `numpy` arrays to perform matrix and vector operations as well as linear algebra.

The first step is performing a test of column-stochasticity. As this means that all the columns sum to one, this can be achieved by testing if the entire vector consists of ones after summing over axis zero of the array corresponding to the matrix. The comparison is done with a tolerance of one machine epsilon, so the difference between 1.0 and the smallest representable 64 bit float larger than one.

The stationary distribution corresponds to the sum-normalized eigenvector for the largest eigenvalue of the matrix. This eigenvalue is one when the matrix is column-stochastic. Numerically, this is retrieved by calculating eigenvalues and -vectors for the matrix with `numpy`, choosing the vector corresponding to the large eigenvalue and dividing it by the sum of the entries of the vector.

The eigenvalues are calculated, as described above, with `numpy` and the second-largest one is returned as μ , g is calculated from it with the relation $g = 1 - \mu$.

For the plots of convergence, the starting point with maximal distance to the stationary distribution of P_0 is chosen, specifically $x_0 = (0, 0, 0, 1)^T$.

Iteration is done by matrix vector multiplication of P_α and x_t .

The variation distance is calculated using the L1 norm $\|x\|_1 = \sum_{i=1}^N |x_i|$ of the difference of x_t and π , the stationary distribution, and dividing it by 2.

Plotting is done with `matplotlib` and the multiplicative factor of μ^t is set to the variation distance at $t = 0$ for the line to start at the same point and make comparisons between the slopes easier.

The results can be replicated by running the python script `numerics_mc.py` with different options, the options are documented when running it with argument `--help`. To get the plots, the wished starting distribution needs to be passed as a list of four space separated values to the option `--x0` and α values can be requested by passing them to the argument `--alphas`. The plots will be written to a folder `results/` that will be created if it does not exist and can optionally be shown directly with the argument `--show`. For replicating the statistical table the option `--conv_stats` together with a number for the randomly generated instances needs to be requested. A seed can also be specified with `--seed`. When doing this the option `--no-save` is also helpful to disable saving of the plots and convergence times for each individual sample. The actual table 2 was generated with 10000 random instances of x_0 .

3.3. Results

This section presents plots for single instance based convergence and tables averaged over multiple instances.

In figure 6 we can see the convergence to stationarity for the original matrix P_0 as defined on page 12. For the first two steps, the L1 distance decreases faster than the theoretical expectation of μ^t but from there on until the end of calculations when the distance is smaller than 1×10^{-5} the slopes of the lines are identical, the the behaviour matches the theoretical expectation.

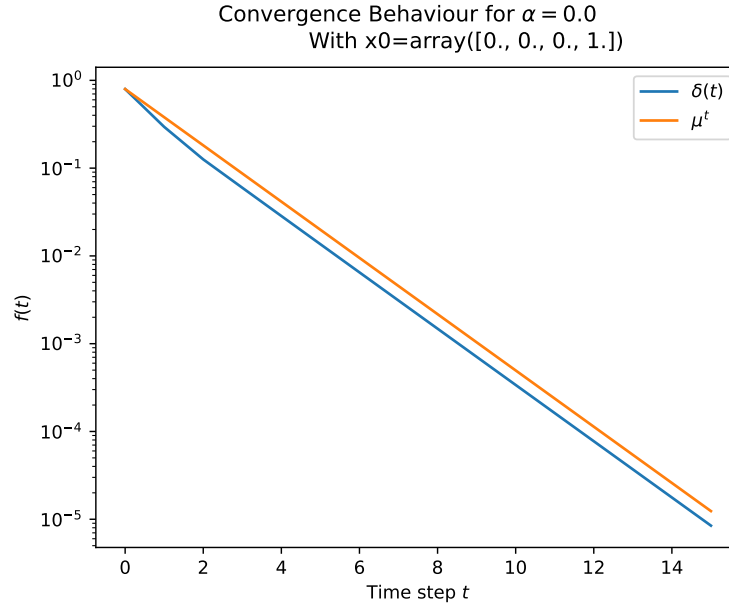


Figure 6. Convergence Behaviour for Original Matrix P_0

352 In figure 7, we can see the convergence when the Markov chain is mixed with the identity
 353 matrix at half contribution from each. Again, the actual convergence dips slightly
 354 below the asymptotic theoretical curve but runs within visible accuracy perfectly to
 355 the theoretical line from approximately $t > 5$. From the diagram it is also visible that
 it takes a bit more than twice as long to reach the final threshold.

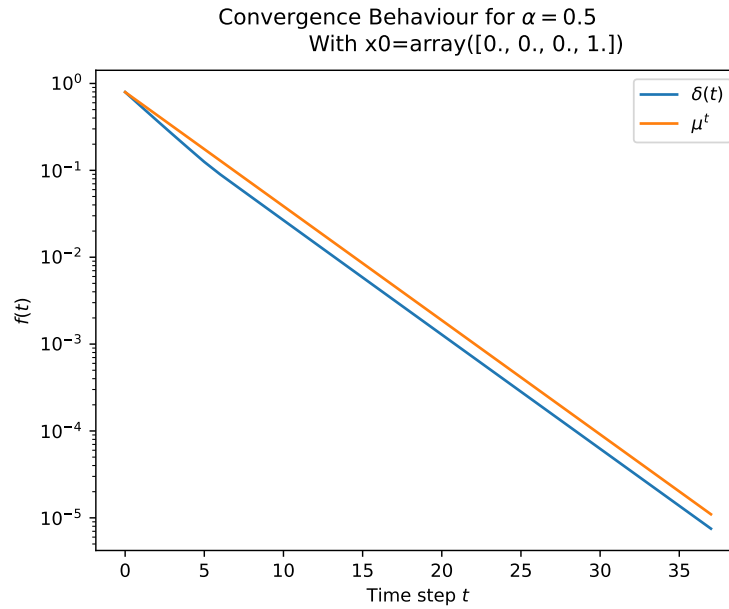


Figure 7. Convergence Behaviour for Lazy Matrix With $\alpha = 0.5$

356

357 When further raising the laziness factor to $\alpha = 0.8$, the convergence behaviour looks
 358 as in figure 8. We can see the same dip and subsequent parallel behaviour as in the
 359 previous plots. The convergence slowed down further and is then a factor of about 3
 360 slower than for $\alpha = 0.5$.

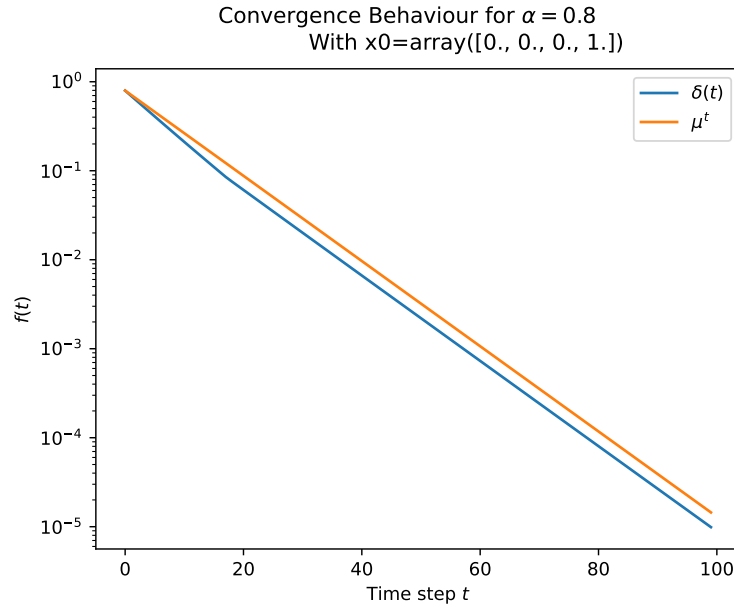


Figure 8. Convergence Behaviour for Lazy Matrix With $\alpha = 0.8$

In table 1 the results of a single convergence for multiple different lazy matrices are shown. The starting condition was the same in all cases and the same as in the figures 6, 7 and 8, so $x_0 = (0, 0, 0, 1)^T$. The table shows the inverse relation between g and the convergence time quite well. For the cases where g is not particularly close to one it holds only approximately. When going from $\alpha = 0$ to 0.5, g is approximately halved while convergence slows down by a bit more than a factor of 2. For $\alpha = 0.8$, g is reduced by a factor of five compared to the original matrix and the convergence time increases more than sixfold. For $\alpha = 0.9$, the spectral gap is halved and the convergence time doubled compared to the previous $\alpha = 0.8$ case. When adding one further nine, so going from 0.9 to 0.99 and from 0.99 to 0.999, the spectral gap is reduced by one order of magnitude and the convergence time increases by approximately the same factor. Especially for the higher end of the laziness factors, the theoretically derived approximate proportionality between $\frac{1}{g}$ and the convergence time holds up under numerical experiments.

Label	μ	g	Time to go below 0.001
$P_{0.0}$	0.478078	0.521922	9
$P_{0.5}$	0.739039	0.260961	21
$P_{0.8}$	0.895616	0.104384	58
$P_{0.9}$	0.947808	0.052192	118
$P_{0.99}$	0.994781	0.005219	1205
$P_{0.999}$	0.999478	0.000522	12069

Table 1. Convergence Table for $x_0 = (0, 0, 0, 1)^T$

Table 2 shows statistics of the convergence speed gathered using 10000 different starting distributions. Each was randomly sampled by taking a vector of four independent random numbers in the unit interval and normalizing the result. This lead to a mean initial variation distance of 0.23 which is between three and four times smaller than the variation distance of $(0, 0, 0, 1)^T$ which explains the smaller mean convergence time in table 2 compared to the previous table 1. Another issue is that the variation distance and the convergence time do not follow the same distribution which makes quantitative comparisons between the single instance and averaged case difficult. However, this is not too relevant here because the main goal is to further support the theoretical finding that the convergence time is approximately inversely related to spectral gap. This finding is well replicated, similarly to the single instance case much more precisely at the upper end of the tested range for α . It is interesting to see how the standard deviation of the convergence behaves almost perfectly linear over the entire range of α values.

Label	g	Mean Convergence Time t_C	Standard Deviation of t_C
$P_{0.0}$	0.521922	6.6118	1.11548
$P_{0.5}$	0.260961	15.7148	2.3642
$P_{0.8}$	0.104384	42.5739	6.07746
$P_{0.9}$	0.052192	87.3741	12.3557
$P_{0.99}$	0.005219	892.331	124.477
$P_{0.999}$	0.000522	8944.04	1260.39

Table 2. Statistics for Convergence for the Different Matrices

4. Conclusion

4.1. Synthesis

- For which values of p do you see high clustering together with short average path length?

We can see an especially large difference between clustering and average shortest path length at $p = 0.01, 0.03, 0.1$. The clustering for these values of p is roughly at 0.95, 0.9 and 0.75, while the average shortest path length has already decreased to 0.55, 0.4 and 0.25 (all values are normalised by the value at $p=0$).

- How variable are the Watts–Strogatz statistics across different random realizations with the same p ?

Looking at Figure 5, the error bars and with this the standard deviation of the values highly depends on the p value. For very low p , the standard deviation of the average shortest path length is very high. The low probability of rewiring an edge leads to graphs with very different shortest paths: No rewiring can happen, which leaves the shortest path length untouched. One or a few very efficient shortcuts, however, can decrease the shortest path length very quickly. We assume this is where the large standard deviation comes from. At higher values of p the graphs get more and more random and the statistical process of rewiring so many edges distributes them so that the shortest paths are very similar in all graphs. For the clustering coefficient we see the highest uncertainty for the higher p values. At low p values only a few connections will be rewired and only by this amount can the clustering coefficient change. When more edges get rewired more connections between neighbours are broken up. The clustering will depend a lot on how exactly the connections are rewired. At $p = 1$ the standard deviation of the clustering is again very small. Again, this is due to the high statistics when rewiring each edge.

- In the Markov-chain experiment, how does the second-largest eigenvalue modulus affect the observed convergence speed?

The second-largest eigenvalue modulus μ determines the speed at which the Markov chain approaches the stationary distribution. When μ is smaller, the term μ^t decreases faster, so the distance between x_t and π becomes small in fewer steps. When μ is closer to 1, μ^t decreases more slowly, so convergence takes longer.

This can be seen in the numerical results. The original matrix P_0 has the smallest value of μ , and it converges the fastest. For the lazy matrices, increasing α moves the nontrivial eigenvalues closer to 1, which therefore increases μ . As a result, the convergence rate increases like this, $P_{0.8} \rightarrow P_{0.5} \rightarrow P_0$. The observed convergence speed agrees with the spectral prediction, in the theoretical section, larger μ means slower convergence.

- Why is the spectral gap $g = 1 - \mu$ a more direct predictor of convergence speed than visual inspection of the entries of P ?

The spectral gap $g = 1 - \mu$ is a more direct predictor of convergence speed because it measures how quickly the slowest non-stationary component of the distribution decays. The entries of P show the one-step transition probabilities, but they do not directly show how repeated multiplication by P behaves over many time steps. Two matrices can look similar when analysed entry by entry, but having different eigenvalues, can result in very different long-term convergence speeds.

The spectral gap is able to summarize this long-term behaviour. If g is large, then μ is far from 1, so the error decreases quickly. If g is small, then μ is close to 1, so the error decreases slowly. For this reason, g is more informative and precise.

- What are the limitations of using only one graph realization for each value of p , or only one initial condition for the Markov chain?

Using only one graph for each value of p is limited since the Watts - Strogatz model is random for $p > 0$. A single graph may not represent the typical behaviour for that value of p , especially for small p , where the presence or absence of only a few shortcuts can strongly affect the average path length.

Similarly, using only one initial condition for the Markov chain is limited because the observed convergence can depend on where the chain starts. Some initial distributions may have a larger component in the slowest-decaying eigenvector direction, while others may have a smaller one. Even if one initial condition is enough to illustrate the spectral mechanism, it does not fully describe all possible convergence behaviours. For a more complete experiment, it would be needed several initial states to make a direct comparison and conclusion of the overall trend.

Overall, scientific research should be done varying all the different parameters that can affect the system at play so a comparison can be done and a long-term behaviour can be studied.

452 **5. Use of LLM tools**

453 For writing the code and report for the part about Watts-Strogatz networks, no LLM
454 was involved.

455 For writing the parts about the Markov chain, both code and in the report, no LLM
456 was involved.

457

Files submitted

458

- `numerics_mc.py`

459

- `Watts_Strogatz_diameter_ASPL.py`

460

- `clustering_coeff.py`

461

- `WS_1_degrees.py`

462

- `WS_2_connected.py`

References

- [1] Fredrik Backåker. The Google Markov Chain: Convergence Speed and Eigenvalues. Technical Report U.U.D.M. Project Report 2012:14, Department of Mathematics, Uppsala University, 2012. <https://uu.diva-portal.org/smash/get/diva2:536076/FULLTEXT01.pdf>.
- [2] Jordi-Lluís Figueras. Lab 03. Watts-Strogatz Networks and Markov-Chain Convergence. Lab handout, Modelling Complex Systems, Spring 2026, 2026. Course material.
- [3] Kranthi Kumar Gade. *Numerical Estimation of the Second Largest Eigenvalue of a Reversible Markov Transition Matrix*. PhD thesis, New York University, 2009. https://cs.nyu.edu/media/publications/gade_kranthi.pdf.
- [4] Eric Roberts and Kelsey Schroeder. The Google PageRank Algorithm. CS 54N Handout #24, Stanford University, <https://web.stanford.edu/class/cs54n/handouts/24-GooglePageRankAlgorithm.pdf>, November 2016.
- [5] ScienceDirect Topics. Markov Chain - an Overview. <https://www.sciencedirect.com/topics/agricultural-and-biological-sciences/markov-chain>. Accessed: 2026-05-12.